00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D

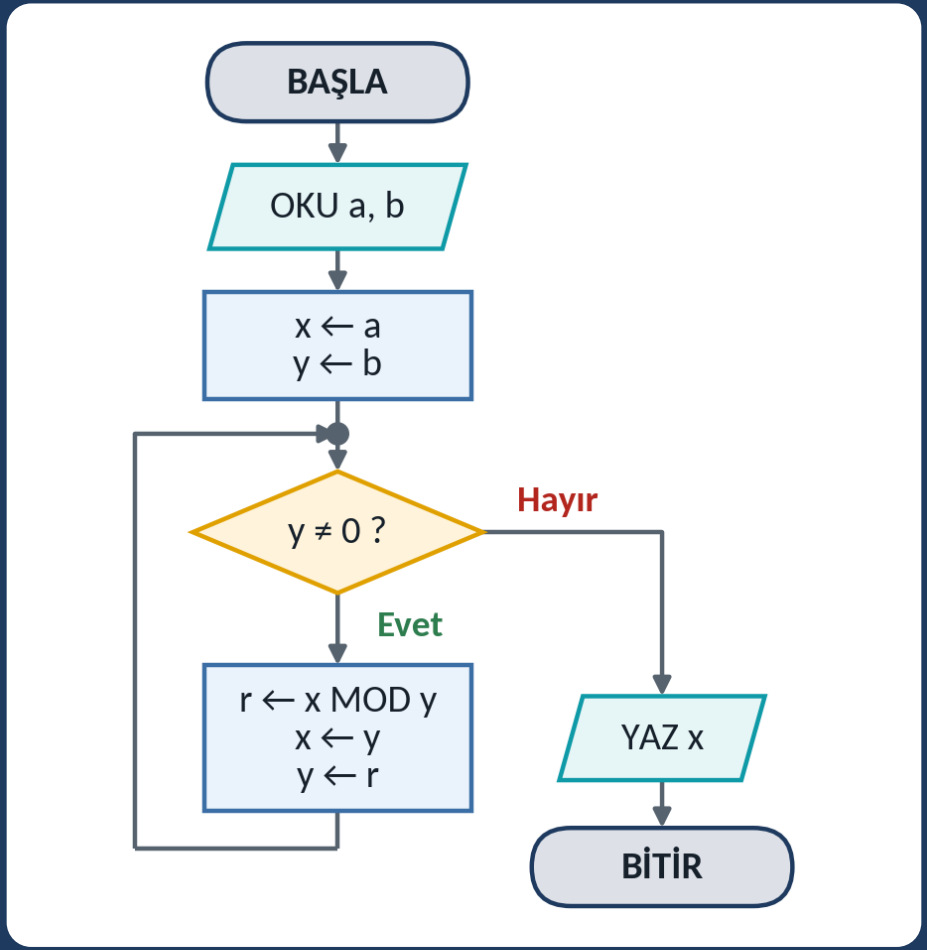
ALGORİTMA VE PROGRAMLAMA – I

HAFTA 02

Algoritmik ve Hesaplamalı Düşünme: Problemden Algoritmaya

Pólya · IPO · sözde kod · akış diyagramı · izleme tablosu · test

MODÜL I · MAKİNE VE ALGORİTMA



Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Güz

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1	H2	H3	H4	H5	H6	H7
Bilgisayar ve Program	**Algoritmik Düşünme**	Değişkenler ve Veri Tipleri	Operatörler ve İfadeler	Karar Yapıları	Döngüler I	Döngüler II + Ara Proje
0x00	0x01	0x02	0x03	0x04	0x05	0x06

H8	H9	H10	H11	H12	H13	H14
Fonksiyonlar I	Kapsam, Yığın, Özyineleme	Diziler I	Diziler II: Matrisler	İşaretçiler ve Bellek	Karakter Dizileri	Yapılar, Dosyalar, Final
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 · Final projesi: H14

Geçen haftadan: üç soru

1. Getir–çöz–yürüt döngüsünde **program sayacı (PC)** hangi bilgiyi tutar?

2. Derleme hattının dört aşamasını sırasıyla yazın: `.c` dosyasından çalıştırılabilir dosyaya.

3. İkili `1011` sayısı onluk tabanda kaçtır? Onluk `13` ikili tabanda nasıl yazılır?

Geçen haftadan: yanıtlar

Program sayacı (PC)

Bir sonra getirilecek komutun bellekteki adresini. Komut getirildikten sonra PC bir sonraki komutu gösterecek biçimde ilerletilir; atlama komutları ise PC'ye yeni bir adres yazar (bugünkü “Kaputun Altında” bölümünün anahtarı).

Derleme hattı

Önişlemci (.c → .i) → derleyici (.i → .s) → çevirici (.s → .o) → bağlayıcı (.o + kütüphaneler → çalıştırılabilir dosya).

İkili sayılar

$1011_2 = 8 + 0 + 2 + 1 = 11$. $13 = 8 + 4 + 1 = 1101_2$. (Bugün bu dönüşümü de bir algoritma olarak düşüneceğiz: sürekli 2'ye böl, kalanları yaz.)

Bir bilgisayara çay demlemeyi nasıl öğretirsiniz?

*"Suyu **kaynat**, **yeterince** çay koy, **biraz** bekle, bardağa koy, **biraz** şeker ekle."*

- Robot: Ne kadar su? Hangi kapta?
- Robot: Kaynadığını nasıl anlayacağım?
- Robot: "Biraz" kaç dakika? Kaç küp?
- Robot: Hangisi önce: çay mı su mu?

HAFTANIN BÜYÜK SORUSU

Bir bilgisayara çay demlemeyi nasıl öğretirsiniz ve **"biraz şeker ekle"** komutu neden bir algoritma adımı olamaz?

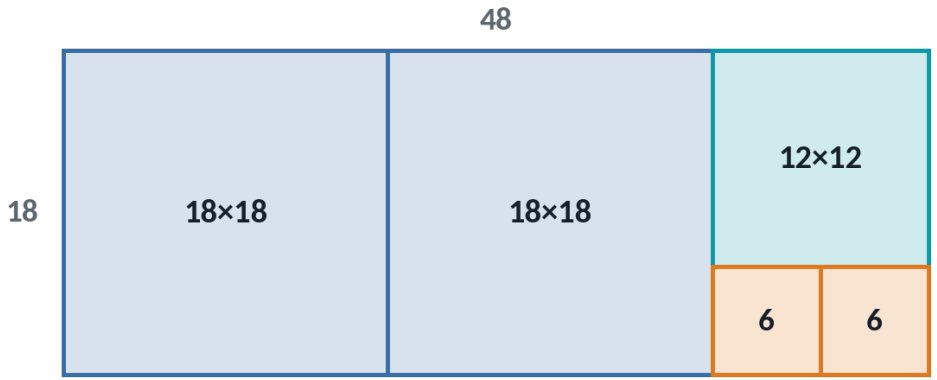
Yanıtı dersin sonunda birlikte vereceğiz.

GERÇEK DÜNYADAN

MÖ ~300 · 9. yüzyıl

“Algoritma”: bir insanın adı ve 2300 yıllık bir yöntem

- **MÖ ~300 · İskenderiye:** Öklid'in “Elemanlar”ında iki sayının **en büyük ortak bölenini** bulan bir yöntem yer alır; bilinen en eski algoritmalardan biridir.
- Fikir: büyük sayıyı küçüğe böl, kalanla devam et; kalan 0 olunca son bölen EBOB'dur (**Örnek 12**).
- **9. yüzyıl · Bağdat: el-Hârizmî**, Hint–Arap rakamlarıyla hesabı anlatan bir eser yazdı; Latince çevirilerde adı **“Algoritmi”** oldu.
- Bu ad zamanla adım adım hesap yöntemlerinin genel adına dönüştü: **algoritma**.



$48 = 2 \cdot 18 + 12 \cdot 18 = 1 \cdot 12 + 6 \cdot 12 = 2 \cdot 6$

EBOB(48, 18) = 6



Kaynak: Knuth (1997), TAOCP Vol. 1, §1.1 · Şekil: 48 × 18 dikdörtgeni artıksız döşeyen en büyük kare 6 × 6

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 2.1 Bir problemi **Girdi–İşlem–Çıktı (IPO)** modeliyle çözümler; kısıtları ve sınır durumlarını belirler.

ÖÇ 2.2 Algoritmanın özelliklerini (**sonluluk, belirlilik, girdi, çıktı, etkinlik**) açıklar ve belirsiz adımları tespit eder.

ÖÇ 2.3 Problemleri **sözde kod** ve **ISO 5807** akış diyagramı sembolleriyle ifade eder.

ÖÇ 2.4 **Sıra, seçim ve tekrar** yapılarının (Böhm–Jacopini) algoritmaları ifade etmeye yettiğini örneklerle gösterir.

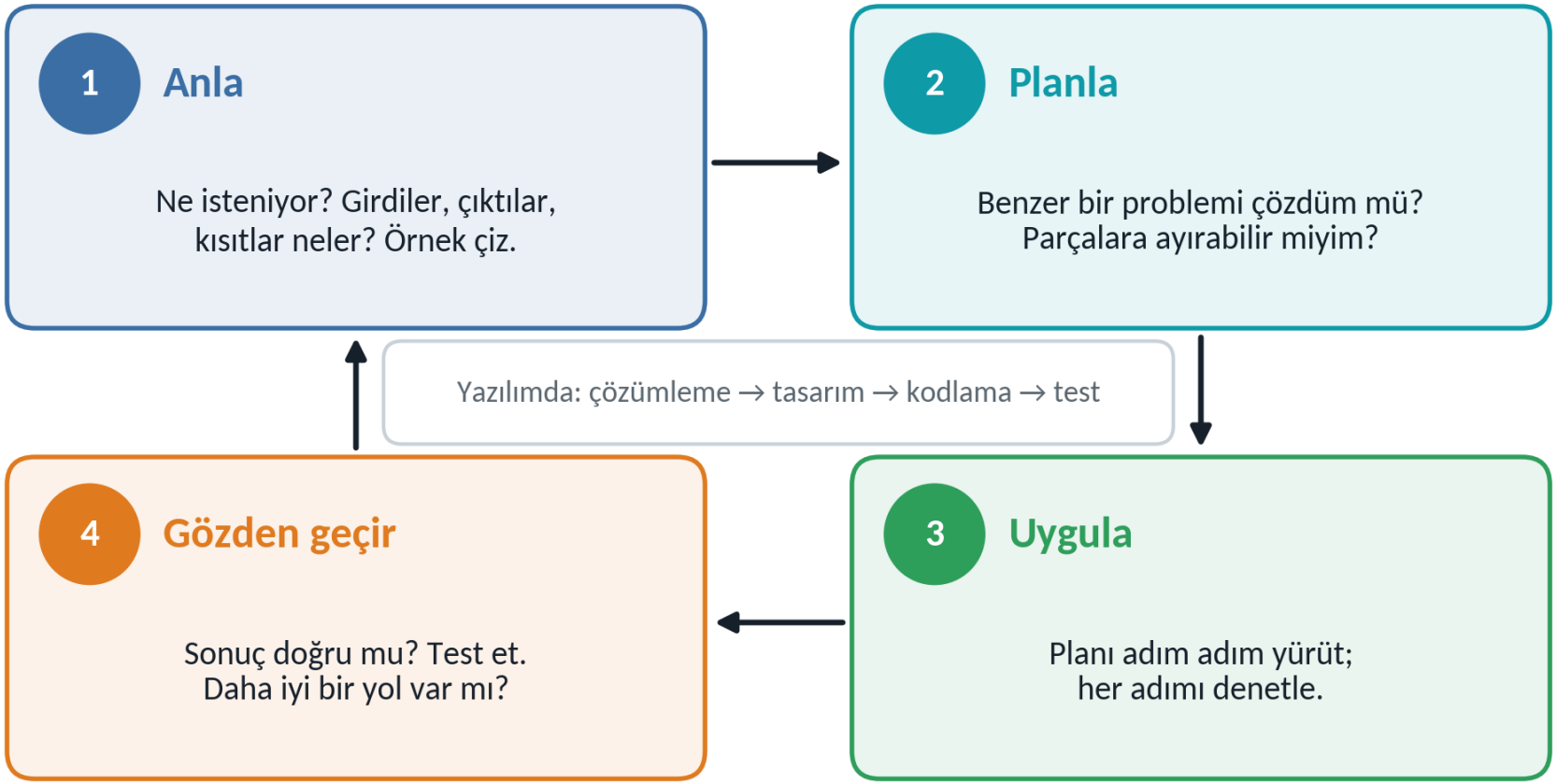
ÖÇ 2.5 Bir algoritmayı **izleme tablosu** ile elle yürütür ve **test durumlarıyla** doğrular.

01

Problemden algoritmaya

Pólya · hesaplamalı düşünme · ayrıştırma · IPO

Pólya'nın dört adımı: anla, planla, uygula, gözden geçir



- **Anla:** girdi, çıktı, kısıt?
- **Planla:** parçalara ayır
- **Uygula:** adım adım yürüt
- **Gözden geçir:** test et, daha iyisi var mı?
- 1 + 2 + … + 100 = ? (Örnek 8)

Pólya (1945), How to Solve It

Hesaplamalı düşünmenin dört bileşeni

Wing (2006): problemleri bir bilgisayarın (ya da insanın) yürütebileceği biçimde ifade etme becerisi, herkes için temel bir beceridir.

Ayrıştırma

- Büyük problemi küçük, çözülebilir parçalara böl.
- **Çay:** suyu kaynat → demle → bardağa koy

Örüntü tanıma

- Parçalar arasındaki benzerlikleri fark et.
- **Çay:** “bekle, denetle, yinele” çok yerde karşımıza çıkar

Soyutlama

- Önemsiz ayrıntıyı at, özü koru.
- **Çay:** çaydanlığın rengi değil, suyun sıcaklığı önemli

Algoritma tasarımı

- Adımları kesin, sıralı ve sonlu biçimde yaz.
- **Çay:** sözde kod + akış diyagramı → C programı

Ayrıştırma: büyük görevi küçük adımlara böl

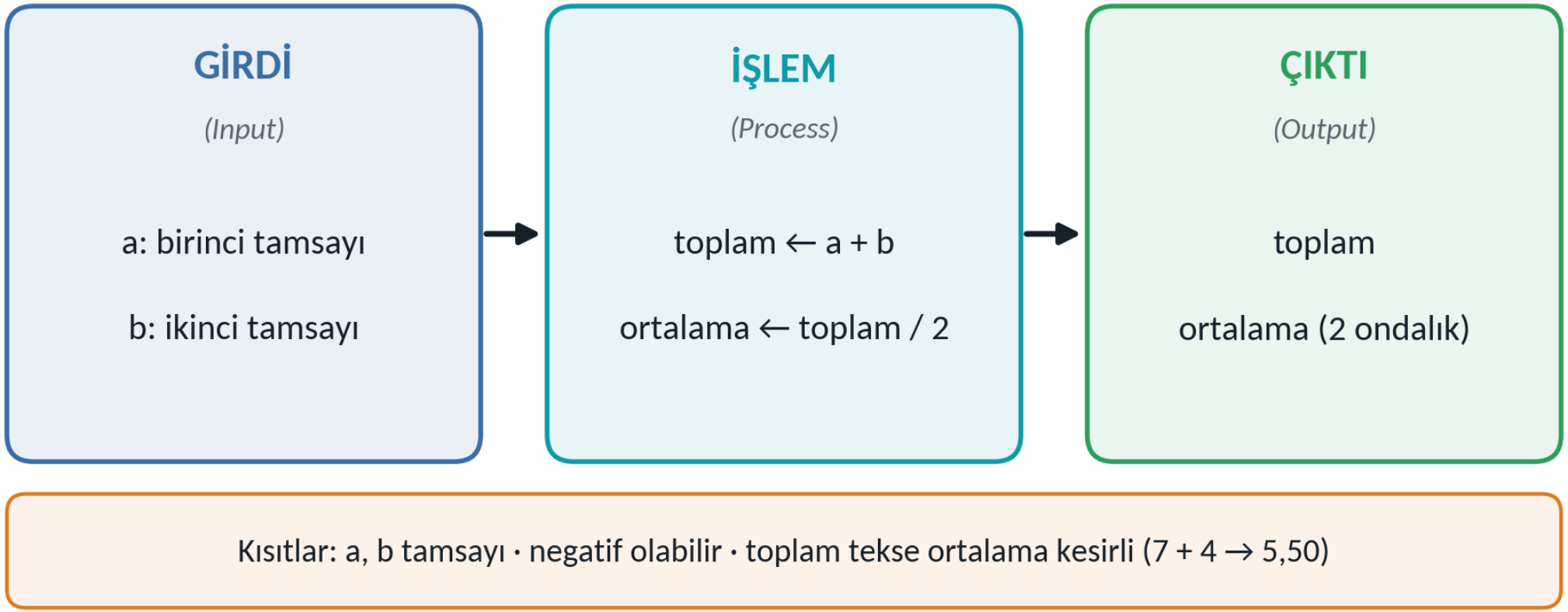


Turuncu yaprak, Büyük Soru'daki alt problemdir; bir sonraki adımda o da ayrıştırılır.

Her yaprak tek başına yapılabilecek kadar küçük olana dek böl (adım adım iyileştirme, Wirth 1971)

Girdi–İşlem–Çıktı (IPO) modeli

Girdi: ne verilecek, hangi aralıkta? · **İşlem:** hangi hesaplar? · **Çıktı:** ne, hangi biçimde? · **Kısıtlar** hataların saklandığı yer



Örnek 2 için IPO tablosu: iki tamsayının toplamı ve ortalaması

02

Algoritma ve ifade biçimleri

Knuth'un beş özelliği · sözde kod · ISO 5807 · değişken ve atama

Knuth: bir algoritmanın beş özelliği

Sonluluk

Sonlu adımdan sonra mutlaka biter. İhlal: `i ← i + 2` iken `i ≠ 10`

Belirlilik

Her adım kesin ve tek anlamlı. İhlal: “biraz şeker ekle”

Girdi

Sıfır ya da daha fazla girdi alır.

Çıktı

En az bir sonuç üretir.

Etkinlik

Her adım kalem–kâğıtla yapılabilecek kadar temel. İhlal: “en iyi hamleyi seç”

Algoritma ≠ program

Algoritma dilden bağımsız bir fikir; program onun C gibi bir dildeki ifadesi.

TAHMİN ET

“Çaya biraz şeker ekle.” Bu adım hangi özelliği ihlal eder?

A Sonluluk: adım hiç bitmez

B Belirlilik: “biraz” kesin ve tek anlamlı değil

C Girdi: şeker bir girdi değildir

D Çıktı: adım bir sonuç üretmez

Aynı algoritmanın dört ifade biçimi

1 · Doğal dil

İki sayı al. Birincisi ikincisinden büyükse birinciyi, değilse ikinciyi ekrana yaz.

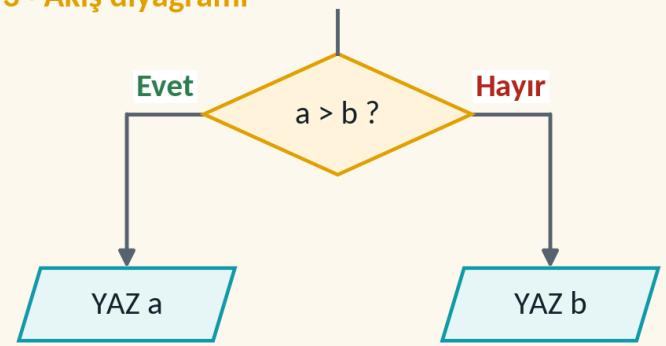
Esnek ama belirsizliğe açık

2 · Sözde kod

```
BAŞLA
OKU a, b
EĞER a > b İSE
    YAZ a
DEĞİLSE
    YAZ b
SON
BİTİR
```

Kesin, dilden bağımsız

3 · Akış diyagramı



Görsel; akışı bir bakışta gösterir

4 · Programlama dili (C)

```
if (a > b) {
    printf("%d\n", a);
} else {
    printf("%d\n", b);
}
```

Derleyicinin anladığı biçim

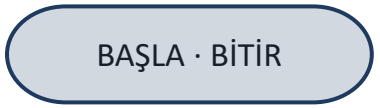
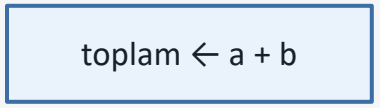
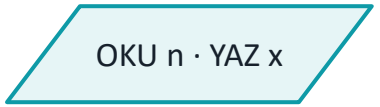
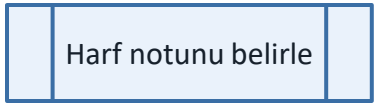
- **Doğal dil:** esnek ama belirsizliğe açık.
- **Sözde kod:** kesin, dilden bağımsız; bu dersin ortak dili.
- **Akış diyagramı:** akışı bir bakışta gösterir.
- **C:** derleyicinin anladığı tek biçim (H3'ten itibaren); çalışan hâli H02_K01_sozlesme_c.c

Sözde kod sözleşmemiz (tüm materyallerde aynı)

Sözde kod	Anlamı	C karşılığı (önizleme)
`BAŞLA … BİTİR`	Algoritmanın başı ve sonu	`int main(void) { … return 0; }`
`OKU x, y`	Kullanıcıdan (klavyeden) değer al	`scanf("%d %d", &x, &y);`
`YAZ ifade, "metin"`	Ekrana değer ya da metin yaz	`printf("%d\n", x);`
`x ← ifade`	Atama: sağ taraf hesaplanır, sonuç x kutusuna yazılır	`x = ifade;`
`=, ≠, <, ≤, >, ≥`	Karşılaştırma (sonuç: doğru/yanlış)	`==, !=, <, <=, >, >=`
`a DIV b, a MOD b`	Tamsayı bölümü ve kalan (a, b ≥ 0)	`a / b, a % b (tamsayılarda)`
`EĞER k İSE … [DEĞİLSE EĞER k2 İSE …] [DEĞİLSE …] SON`	Seçim: koşula göre dallanma	`if (k) { … } else if (k2) { … } else { … }`
`İKEN k YAP … SON`	Tekrar: koşul doğru oldukça gövdeyi yinele (önce sına)	`while (k) { … }`
`İÇİN i ← 1'DEN n'YE YAP … SON`	Sayaçlı tekrar: i ← 1; İKEN i ≤ n … i ← i + 1 kısaltması	`for (int i = 1; i <= n; i = i + 1) { … }`
`// açıklama`	Okuyucu için not; yürütülmez	`/* açıklama */`

Girinti: 4 boşluk · her yapı SON ile kapanır · ← atama, = karşılaştırma · C karşılıklarının çalışan hâli: H02_K01_sozlesme_c.c

Akış diyagramı sembolleri (ISO 5807)

Sembol	Ad	Anlamı
	**Başla / Bitir** *terminal*	Algoritmanın giriş ve çıkış noktası
	**İşlem** *process*	Hesaplama veya atama
	**Giriş / Çıkış** *data (I/O)*	Veri okuma ya da sonuç yazma
	**Karar** *decision*	Koşul sınanır; Evet/Hayır dalına gidilir
	**Önceden tanımlı işlem** *predefined process*	Başka yerde tanımlı alt algoritma
	**Bağlayıcı** *connector*	Kesilen akışı başka noktada sürdürür
	**Akış oku** *flow line*	Yürütme sırasını gösterir (yukarıdan aşağıya)

Çizim kuralları

- Akış **yukarıdan aşağıya**.
- Her karar elmasının iki dalı: **Evet / Hayır**.
- Tek BAŞLA, en az bir BİTİR.
- Oklar kesişmez; birleşme noktasında (•) buluşur.
- Sayfa bitince **bağlayıcı** (A → A).

Değişken = etiketli kutu; atama = sağdan sola



Değişken = adı olan bir kutu; içinde her an tek bir değer bulunur.

"←" sağdan sola okunur. Matematikte "x = x + 1" çelişkidir;
algoritmada "x ← x + 1" yalnızca x'i bir artırır.

toplam ← toplam + i: önce sağ taraf hesaplanır (10 + 5), sonra sonuç sol kutuya yazılır; eski değer silinir

TAHMİN ET

Bu sözde kod ne yazdırır?

```
1  x ← 3
2  y ← x
3  x ← x + y
4  YAZ x, y
```

A 6, 6

B 6, 3

C 3, 6

D 3, 3

03

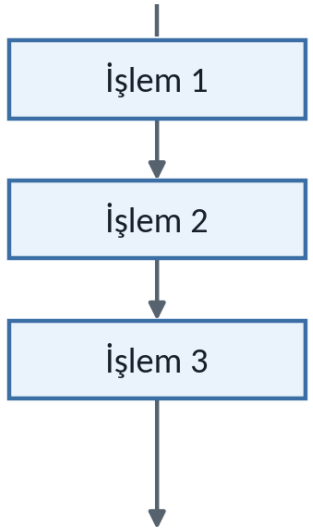
Kontrol yapıları, izleme ve test

Sıra · seçim · tekrar · işlemcide karar · izleme tablosu · sınır değerleri

Üç yapı yeter: sıra, seçim, tekrar

SIRA

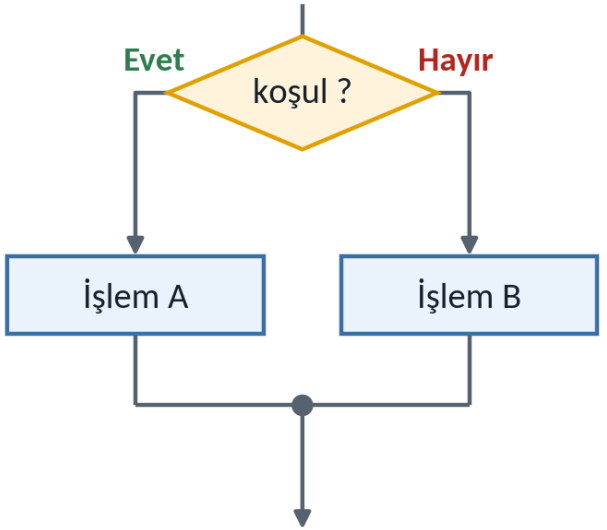
sequence




```
işlem 1
işlem 2
işlem 3
```

SEÇİM

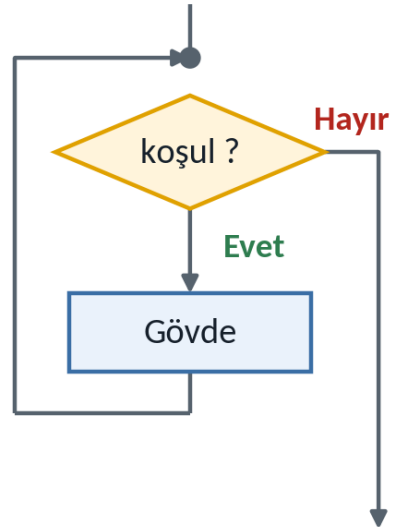
selection




```
EĞER koşul İSE
    işlem A
DEĞİLSE
    işlem B
SON
```

TEKRAR

iteration




```
İKEN koşul YAP
    gövde
SON
```

Böhm ve Jacopini (1966): her akış diyagramı bu üç yapıyla ifade edilebilir (yapısal program teoremi)

KAPUTUN ALTINDA

Karar elması işlemcide: karşılaştır + koşullu atla

- İşlemcinin “karar” komutu yok: ◇ = `cmp` + `jle`.
- `cmp` bayrakları ayarlar; `jle` bayraklara bakıp **PC'ye yeni adres** yazar.
- Döngü = **geriye doğru atlama** (`jle .L3`).
- Her tur: gövde 3 + koşul 3 = **6 komut**.
- N = 1000 → yalnız döngü ≈ **6 000 komut**.

gcc -O0 -S -masm=intel · H02_07_toplam_dongu.c · döngü bölümü

```
    mov DWORD PTR [rbp-16], 0   ; toplam ← 0
    mov DWORD PTR [rbp-12], 1   ; i ← 1
    jmp .L2                     ; önce koşula git
.L3:                            ; döngü gövdesi
    mov eax, DWORD PTR [rbp-12] ; eax ← i
    add DWORD PTR [rbp-16], eax ; toplam ← toplam + i
    add DWORD PTR [rbp-12], 1   ; i ← i + 1
.L2:                            ; koşul
    mov eax, DWORD PTR [rbp-20] ; eax ← n
    cmp DWORD PTR [rbp-12], eax ; i ile n'yi karşılaştır
    jle .L3                     ; i ≤ n ise GERİ atla
```

Sarı etiketler atlama hedefleri · vurgulu iki satır = karar elması (◇) · `jle .L3` yukarı atlar = döngü

İzleme tablosu: algoritmayı kâğıtta yürüt

Her **değişkene**, **koşula** ve **çıktıya** bir sütun · değer almamış kutu: **–** · yalnızca **o adımda değişen** hücreyi yaz ve işaretle

Örnek 7, n = 3 (değer almamış kutu: –)

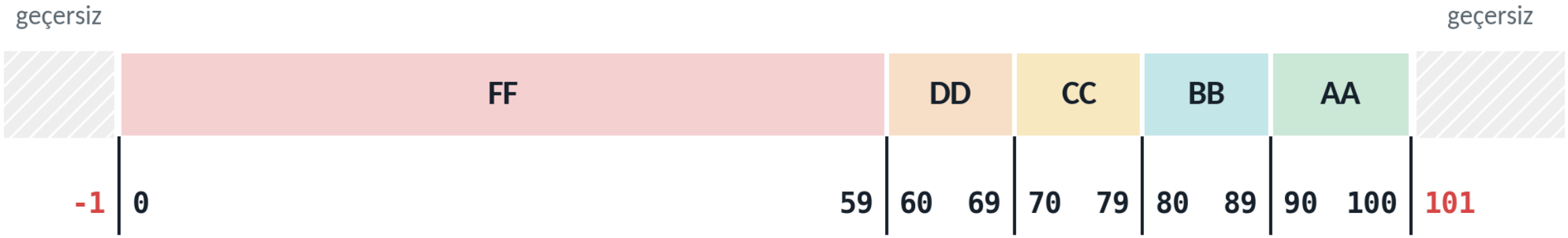
değeri o adımda değişen hücre

		değişkenler			koşul	çıktı
Adım	**İşlem**	**n**	**toplam**	**i**	**Koşul i ≤ n**	**Çıktı**
1	OKU n	**3**	–	–		
2	toplam ← 0	3	**0**	–		
3	i ← 1	3	0	**1**		
4	i ≤ n ?	3	0	1	**1 ≤ 3 → Evet**	
5	toplam ← toplam + i	3	**1**	1		
…	…	…	…	…	…	…

Şablon: Örnek 7 (1..N toplamı), n = 3 için ilk beş adım; sarı hücreler o adımda değişen değerler

Test: normal, sınır ve geçersiz girdiler

Sınır değer testi: her aralığın iki kenarını ve hemen dışını dene



Her dikey çizgi bir sınırdır: hemen solundaki ve sağındaki değerler ayrı birer test durumudur (kırmızı: geçersiz girdi).

Test durumu

girdi + **beklenen** çıktı; beklenen değeri önce elle hesapla

Normal

85 → BB

Sınır

89 / 90 · 59 / 60 · 0 · 100

Geçersiz

−1 · 101 → “Geçersiz”

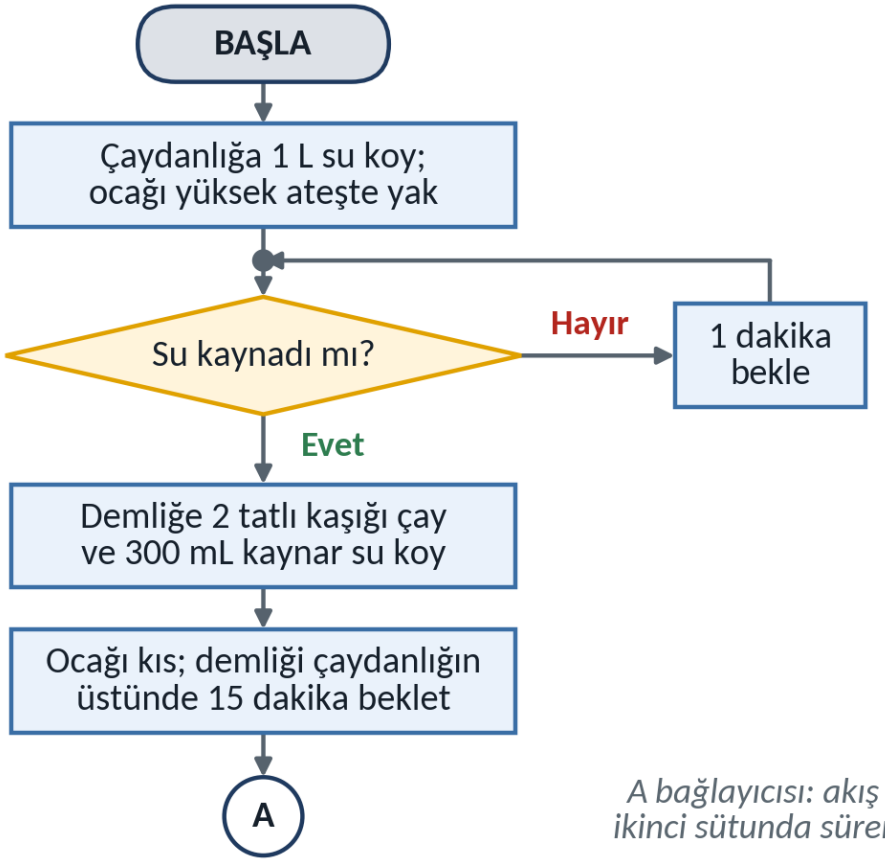
Hataların çoğu sınırlarda saklanır: programda < mı yazılmalıydı, ≤ mü? Tek bir karakter.

04

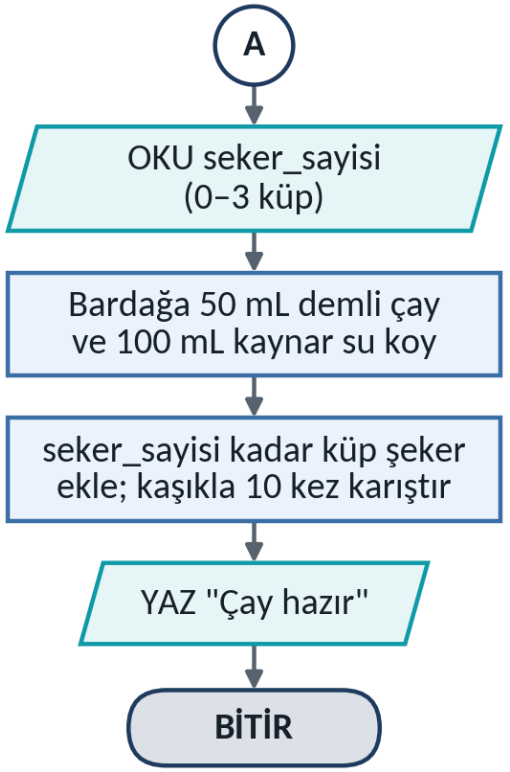
Örnek merdiveni

12 basamak · her basamak tek bir yeni öge · C yalnızca önizleme

Örnek 1 ★ — Çay demleme algoritması



A bağlayıcısı: akış ikinci sütunda sürer



- "Su kaynayınca" → **tekrar**: kaynamadıysa 1 dk bekle, yeniden bak.
- "Biraz şeker" → **girdi**: OKU seker_sayisi (0–3).
- "Biraz bekle" → **15 dakika**.
- Akış iki sütuna sığmadı: **A bağlayıcısı**.
- Değiştir: 5 küp istenirse ne olmalı?

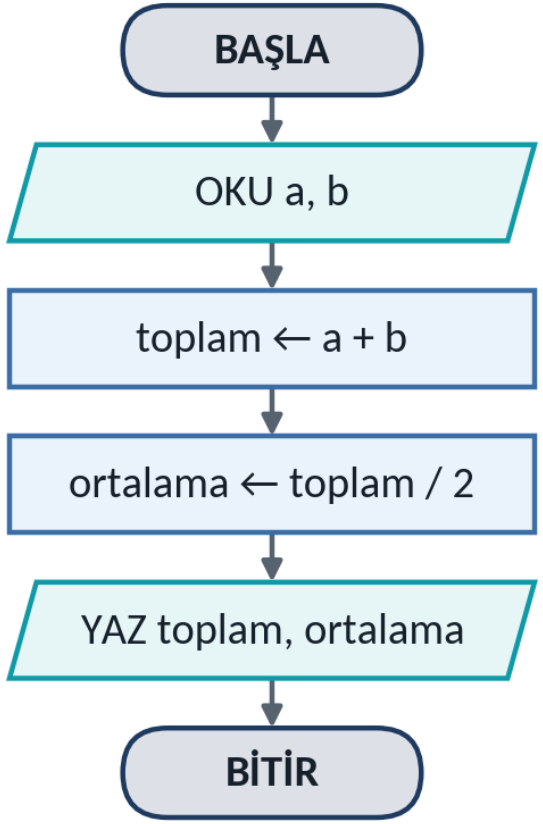
Örnek 2 ★ — İki sayının toplamı ve ortalaması (sıra)

SÖZDE KOD

```
BAŞLA
OKU a, b
toplam ← a + b
ortalama ← toplam / 2
YAZ toplam, ortalama
BİTİR
```

GERÇEK ÇIKTI

```
$ echo "7 4" | ./toplam_ortalama
İki tamsayı girin:
Toplam   = 11
Ortalama = 5.50
```



H02_02_toplam_ortalama.c · satır 13–16

```c
13      scanf("%d %d", &a, &b);          /* OKU a, b                */
14
15      int toplam = a + b;              /* toplam ← a + b          */
16      double ortalama = toplam / 2.0;  /* ortalama ← toplam / 2 */
```

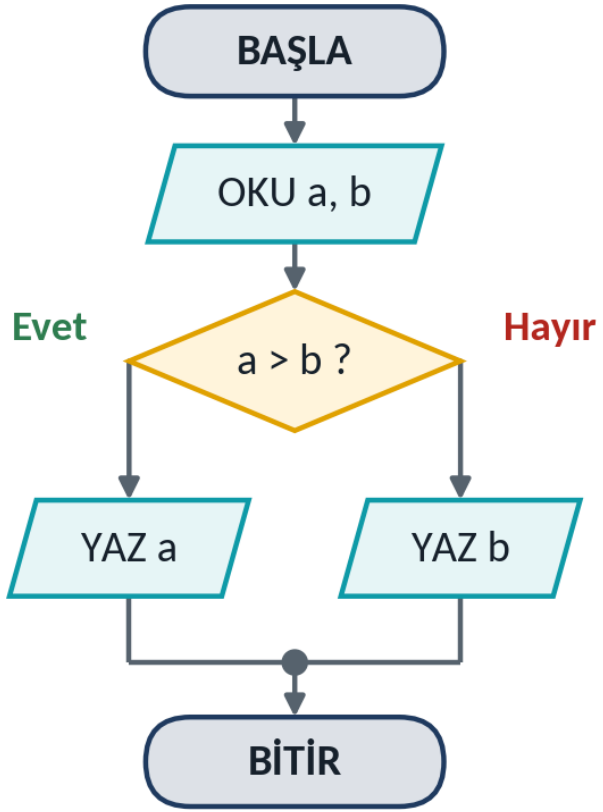
Örnek 3 ★ — İki sayının büyüğü (seçim)

SÖZDE KOD

```
BAŞLA
OKU a, b
EĞER a > b İSE
    YAZ a
DEĞİLSE
    YAZ b
SON
BİTİR
```

GERÇEK ÇIKTI

```
$ echo "5 5" | ./iki_sayi_buyuk
İki tamsayı girin:
Büyük olan: 5
```



H02_03_iki_sayi_buyuk.c · satır 15–19

```
15      if (a > b) {                    /* EĞER a > b İSE */
16          printf("Büyük olan: %d\n", a);
17      } else {                        /* DEĞİLSE */
18          printf("Büyük olan: %d\n", b);
19      }                               /* SON */
```

Örnek 4 ★ — Tek mi çift mi? (kalan işlemi)

SÖZDE KOD

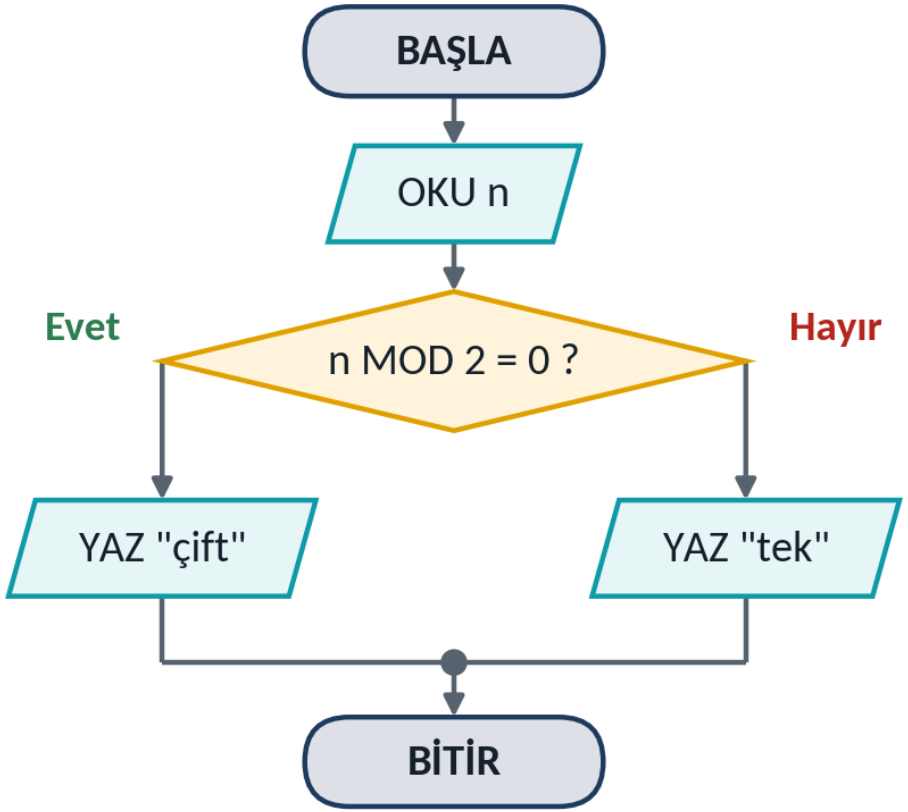
```
BAŞLA
OKU n
EĞER n MOD 2 = 0 İSE
    YAZ "çift"
DEĞİLSE
    YAZ "tek"
SON
BİTİR
```

GERÇEK ÇIKTI

```
$ echo "-7" | ./tek_cift
Bir tamsayı girin:
-7 tek sayıdır
```

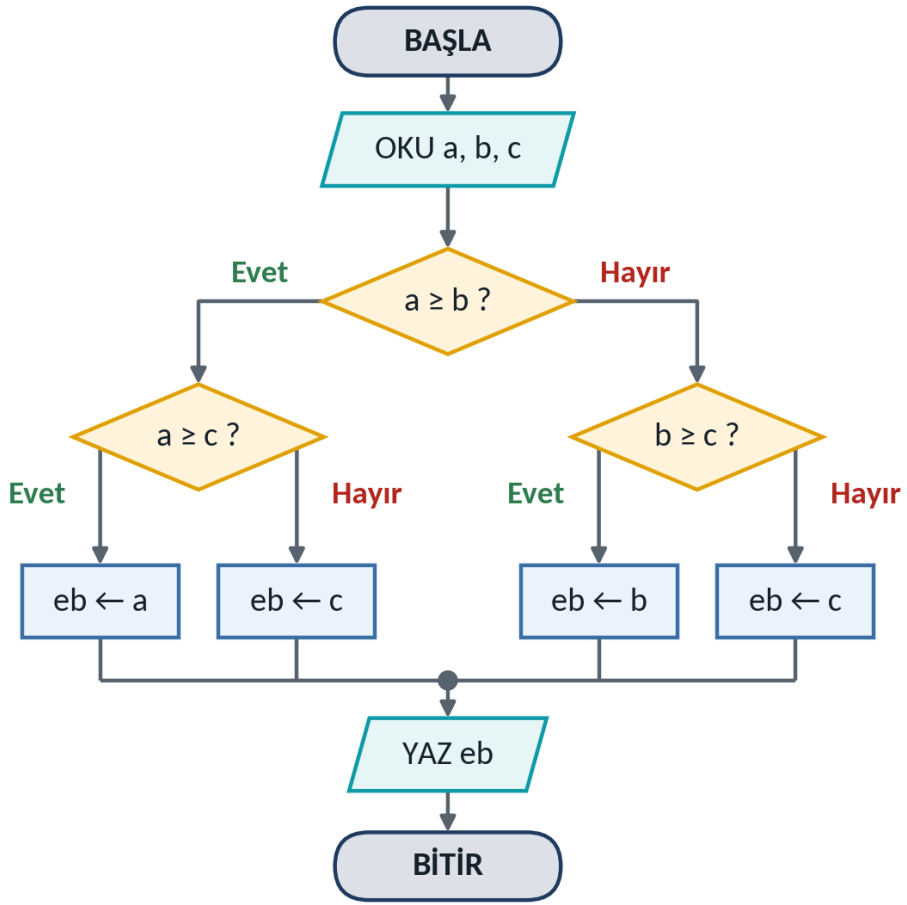
H02_04_tek_cift.c · satır 14–18

```c
14      if (n % 2 == 0) {     /* EĞER n MOD 2 = 0 İSE */
15          printf("%d çift sayıdır\n", n);
16      } else {
17          printf("%d tek sayıdır\n", n);
18      }
```

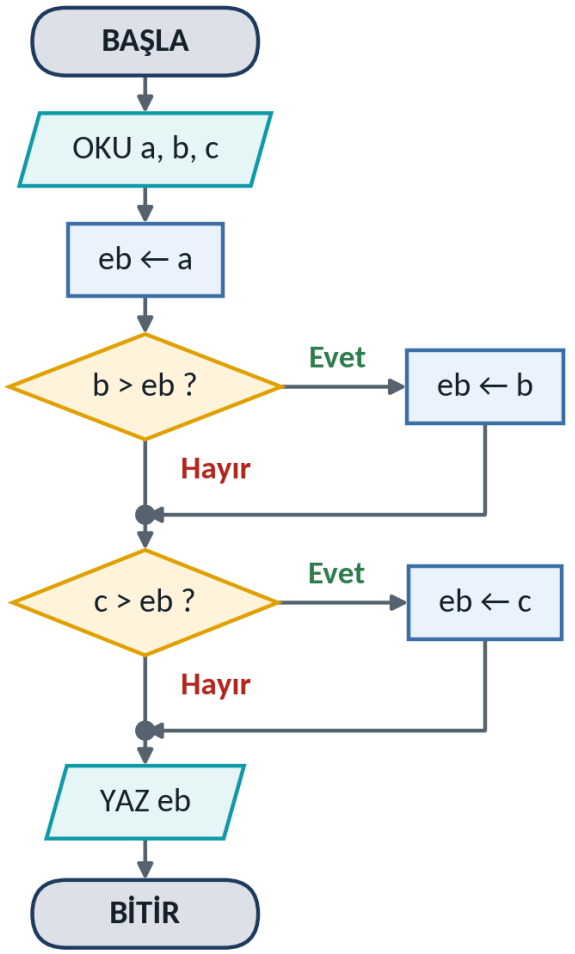


Örnek 5 ★★ — Üç sayının en büyüğü: iki strateji

Strateji 1: iç içe karar

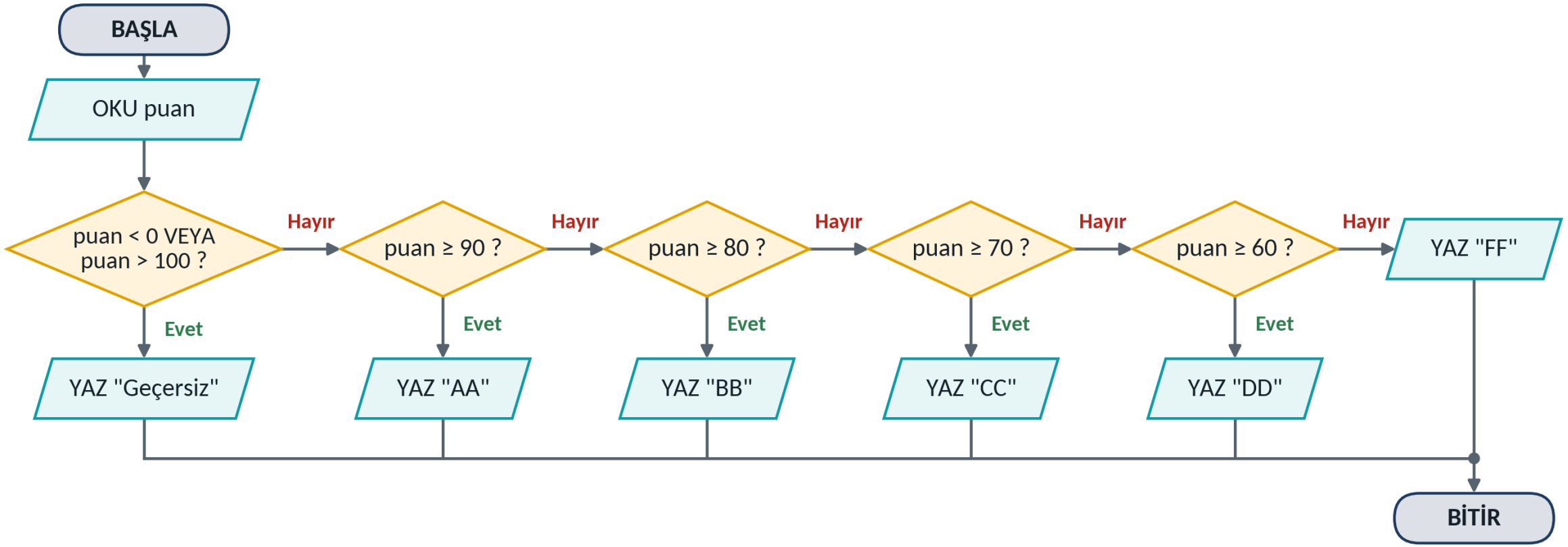


Strateji 2: ardışık güncelleme



- İkisi de **2 karşılaştırma** yapar.
- Strateji 1: karar ağacı; 4 sayı için **7 elmas**.
- Strateji 2: “şimdiye kadarki en büyük”; 4 sayı için **bir EĞER** daha.
- N sayı → strateji 2 **döngü** olur (Örnek 9).
- Gerçek çıktı: 4 9 7 → 9 · −5 −2 −8 → −2

Örnek 6 ★★ — Puandan harf notu (karar merdiveni)



Koşullar büyükten küçüğe sınanır: "puan ≥ 80" sorulduğunda puanın 90'dan küçük olduğu zaten bellidir (örnek ölçek)

Örnek 6 — C önizlemesi ve sınır değer testleri

H02_06_harf_notu.c · satır 15–27

```c
15      if (puan < 0 || puan > 100) {
16          printf("Geçersiz puan: %d\n", puan);
17      } else if (puan >= 90) {
18          printf("Harf notu: AA\n");
19      } else if (puan >= 80) {
20          printf("Harf notu: BB\n");
21      } else if (puan >= 70) {
22          printf("Harf notu: CC\n");
23      } else if (puan >= 60) {
24          printf("Harf notu: DD\n");
25      } else {
26          printf("Harf notu: FF\n");
27      }
```

Test	Girdi	Beklenen	Program	
Normal	85	BB	BB	✓
Sınır (üst uç)	100	AA	AA	✓
Sınır	90	AA	AA	✓
Sınır	89	BB	BB	✓
Sınır	60	DD	DD	✓
Sınır	59	FF	FF	✓
Sınır (alt uç)	0	FF	FF	✓
Geçersiz	-5	Geçersiz	Geçersiz puan: -5	✓
Geçersiz	101	Geçersiz	Geçersiz puan: 101	✓

“Program” sütunu gerçek çalıştırmalardan (kodlar/ciktilar/H02_06_harf_notu.txt)

Örnek 7 ★★ — 1'den N'e kadar toplam (tekrar)

SÖZDE KOD

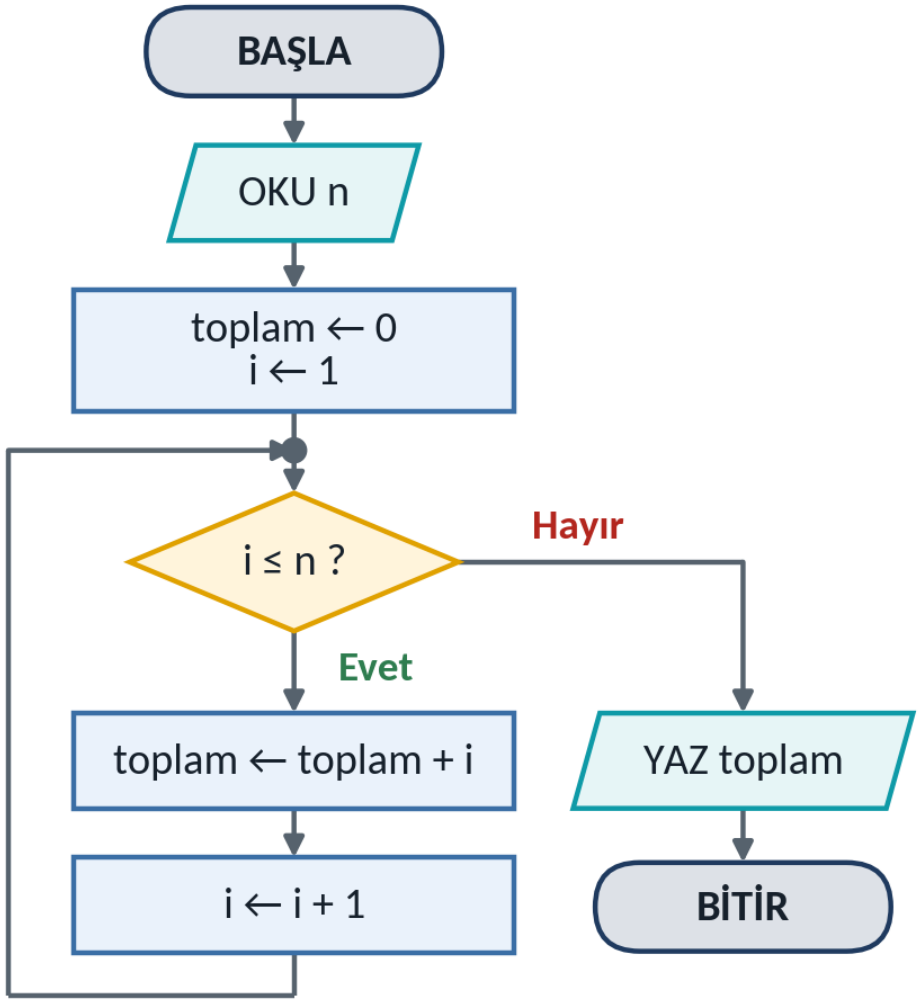
```
BAŞLA
OKU n
toplam ← 0      // toplayıcı
i ← 1           // sayaç
İKEN i ≤ n YAP
    toplam ← toplam + i
    i ← i + 1
SON
YAZ toplam
BİTİR
```

GERÇEK ÇIKTI

```
$ echo "3" | ./toplam_dongu
N değerini girin:
Toplam (1..3) = 6
```

H02_07_toplam_dongu.c · satır 15–20

```
15      int toplam = 0;             /* toplam ← 0          */
16      int i = 1;                  /* i ← 1               */
17      while (i <= n) {            /* İKEN i ≤ n YAP      */
18          toplam = toplam + i;    /*   toplam ← toplam + i */
19          i = i + 1;              /*   i ← i + 1         */
20      }                           /* SON                 */
```



Örnek 7 — izleme tablosu (n = 3)

SÖZDE KOD

```
BAŞLA
OKU n
toplam ← 0      // toplayıcı
i ← 1           // sayaç
İKEN i ≤ n YAP
    toplam ← toplam + i
    i ← i + 1
SON
YAZ toplam
BİTİR
```

Adım	İşlem	n	toplam	i	i ≤ n ?	Çıktı
1	OKU n	**3**	–	–		
2	toplam ← 0	3	**0**	–		
3	i ← 1	3	0	**1**		
4	i ≤ n ?	3	0	1	**1 ≤ 3 → Evet**	
5	toplam ← toplam + i	3	**1**	1		
6	i ← i + 1	3	1	**2**		
7	i ≤ n ?	3	1	2	**2 ≤ 3 → Evet**	
8	toplam ← toplam + i	3	**3**	2		
9	i ← i + 1	3	3	**3**		
10	i ≤ n ?	3	3	3	**3 ≤ 3 → Evet**	
11	toplam ← toplam + i	3	**6**	3		
12	i ← i + 1	3	6	**4**		
13	i ≤ n ?	3	6	4	**4 ≤ 3 → Hayır**	
14	YAZ toplam	3	6	4		**6**

TAHMİN ET

Bu algoritma n = 4 için ne yazdırır?

```
1  toplam ← 0
2  i ← 1
3  İKEN i < n YAP
4      toplam ← toplam + i
5      i ← i + 1
6  SON
7  YAZ toplam
```

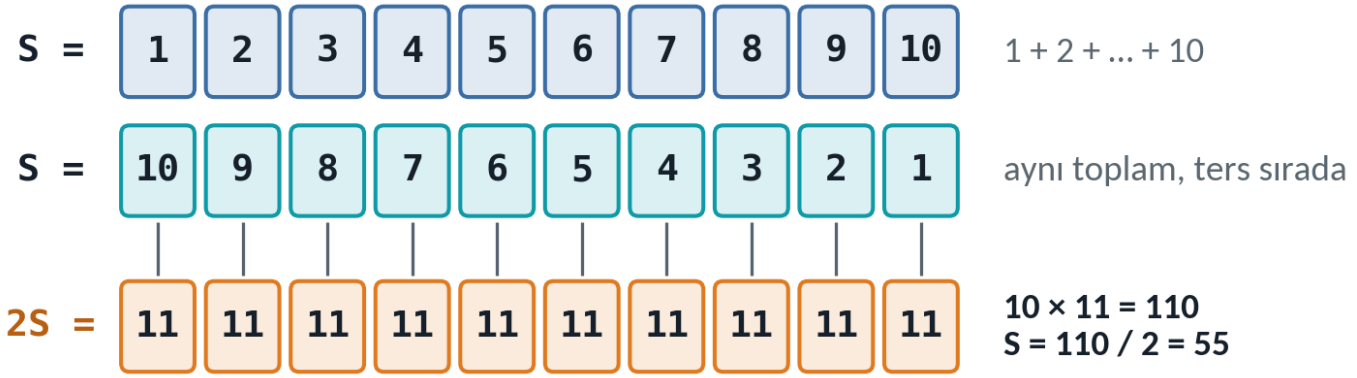
A 10

B 6

C 4

D Sonsuz döngü

Örnek 8 ★★ — Aynı toplam, Gauss formülüyle



Genel olarak S = N × (N + 1) / 2: N ne olursa olsun yalnızca 3 işlem

H02_08_gauss_toplam.c · satır 17–25

```c
17      int toplam = 0, i = 1, tekrar = 0;
18      while (i <= n) {
19          toplam = toplam + i;
20          i = i + 1;
21          tekrar = tekrar + 1;
22      }
23
24      /* Algoritma B: Gauss formülü, tekrar yok */
25      int gauss = n * (n + 1) / 2;
```

GERÇEK ÇIKTI

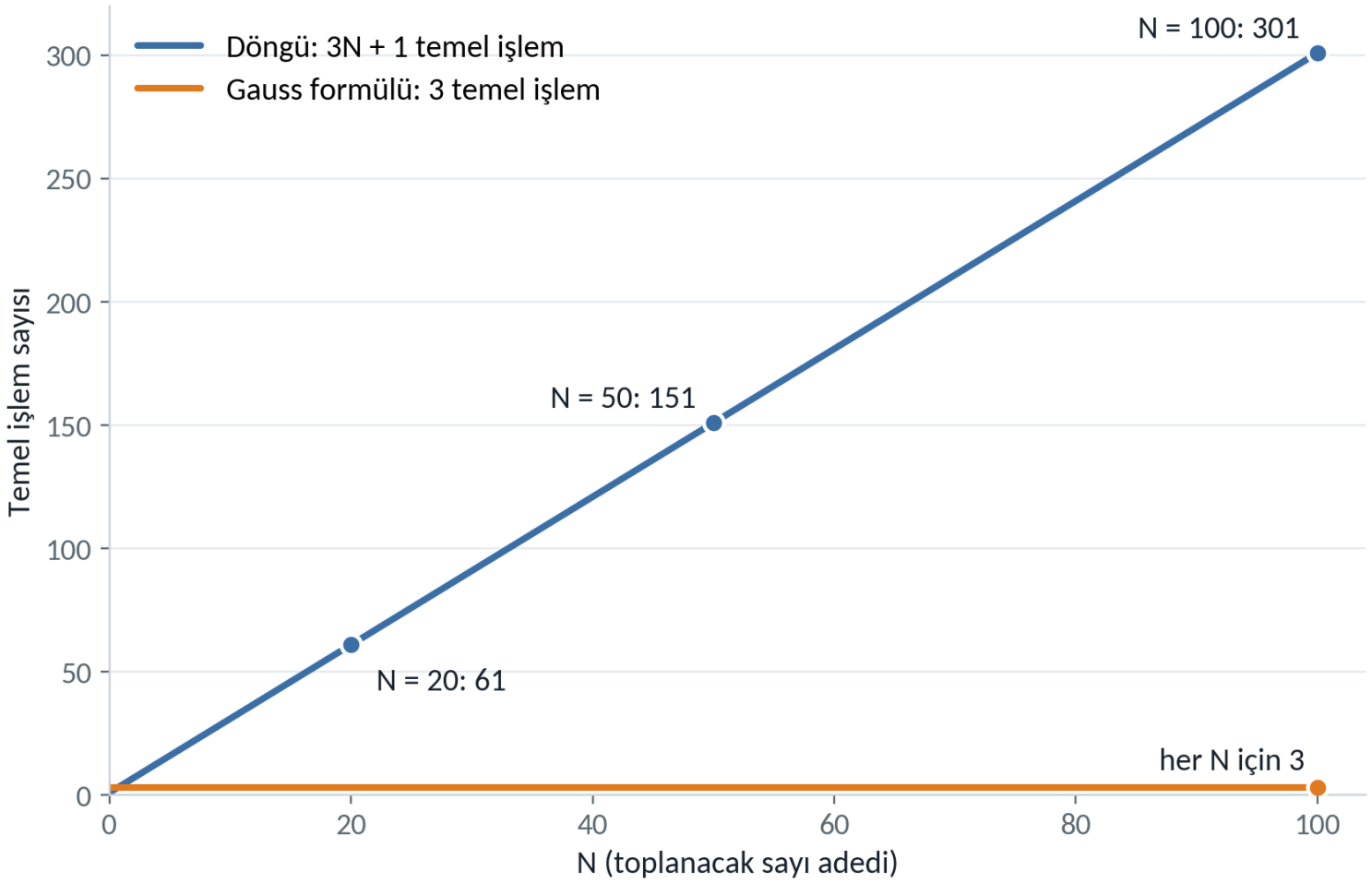
```
$ echo "100" | ./gauss_toplam
N değerini girin:
A (döngü): toplam = 5050, 100 tur
B (Gauss): toplam = 5050, tur yok

$ echo "10000" | ./gauss_toplam
N değerini girin:
A (döngü): toplam = 50005000, 10000 tur
B (Gauss): toplam = 50005000, tur yok
```

SÖZDE KOD

```
BAŞLA
OKU n
toplam ← n × (n + 1) / 2
YAZ toplam
BİTİR
```

Adım sayısı: 3N + 1 işlem mi, 3 işlem mi?

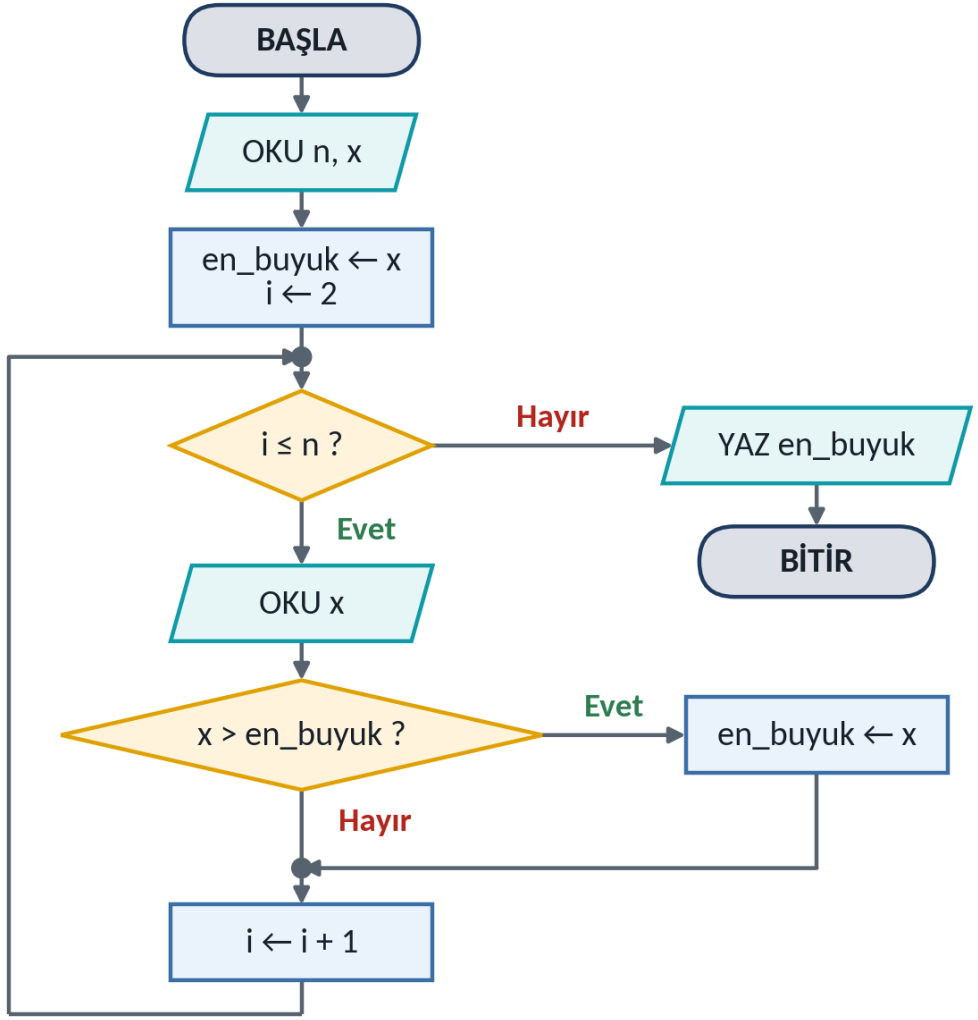


- Döngü her turda: **karşılaştır, topla, artır** → 3N + 1
- Formül: **1 toplama, 1 çarpma, 1 bölme** → 3
- İşlemcide (Örnek 8, gerçek -O0 çıktısı): döngü **7 komut/tur** (gövde 4, koşul 3); formül **9 komut**
- İkisi de **doğru**; biri N ile büyür, diğeri sabit.

Örnek 9 ★★ — N sayının en büyüğü (tekrar + seçim)

SÖZDE KOD

```
BAŞLA
OKU n, x
en_buyuk ← x   // ilk aday
i ← 2
İKEN i ≤ n YAP
    OKU x
    EĞER x > en_buyuk İSE
        en_buyuk ← x
    SON
    i ← i + 1
SON
YAZ en_buyuk
BİTİR
```



Örnek 9 — izleme tablosu, C önizlemesi ve testler

i	okunan x	x > en_buyuk ?	en_buyuk
başlangıç	34	–	**34**
2	12	**12 > 34 → Hayır**	34
3	58	**58 > 34 → Evet**	**58**
4	7	**7 > 58 → Hayır**	58
5	41	**41 > 58 → Hayır**	58

Test	Girdi	Beklenen	Program	
Normal	5 34 12 58 7 41	58	58	✓
Tümü negatif	3 -5 -2 -9	-2	-2	✓
Tek sayı	1 42	42	42	✓
Tümü eşit	4 7 7 7 7	7	7	✓

Girdi: n = 5 · 34 12 58 7 41 — en_buyuk yalnızca bir kez değişti (58)

H02_09_n_sayi_en_buyuk.c · satır 18–26

```
18      int en_buyuk = x;   /* ilk sayı: en iyi aday */
19      int i = 2;
20      while (i <= n) {
21          scanf("%d", &x);
22          if (x > en_buyuk) {   /* daha büyükse güncelle */
23              en_buyuk = x;
24          }
25          i = i + 1;
26      }
```

Örnek 10 ★★ — Faktöriyel: başlangıç değeri 1

SÖZDE KOD

```
BAŞLA
OKU n
carpim ← 1     // 0 değil!
İÇİN i ← 1'DEN n'YE YAP
    carpim ← carpim × i
SON
YAZ carpim
BİTİR
```

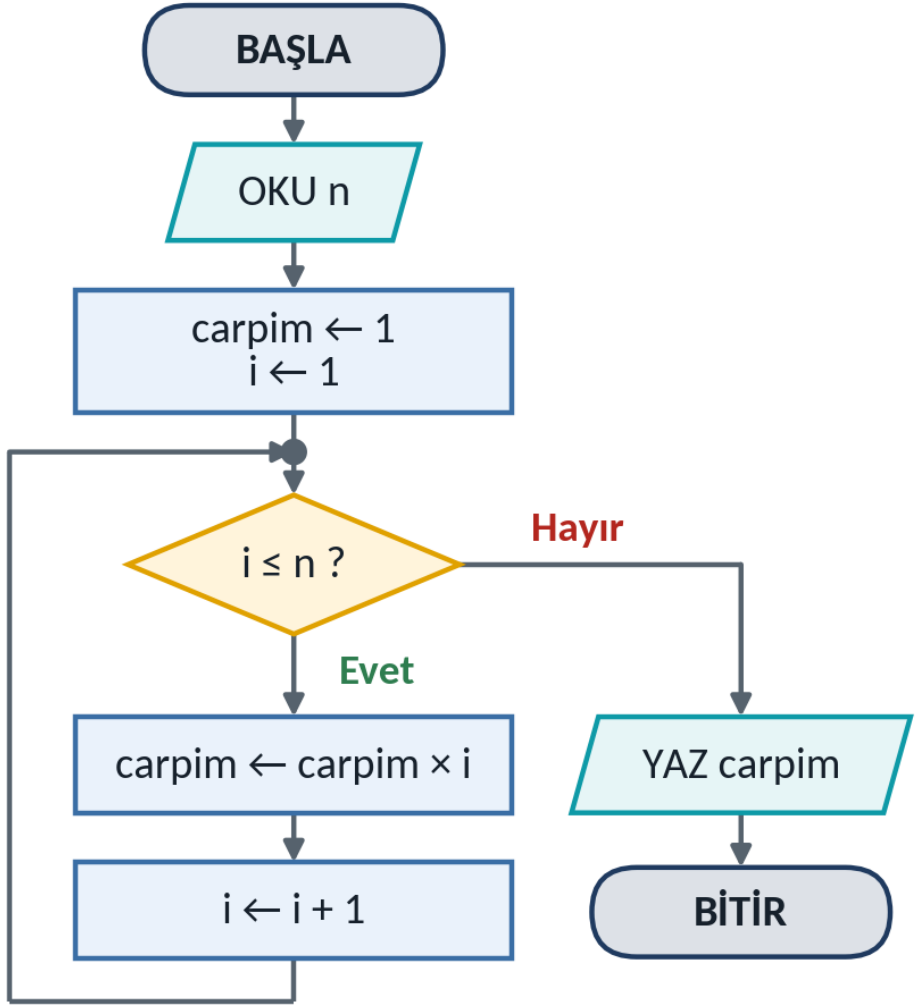
GERÇEK ÇIKTI

```
$ echo "5" | ./faktoriyel
n değerini girin (0-12):
5! = 120

$ echo "0" | ./faktoriyel
n değerini girin (0-12):
0! = 1
```

H02_10_faktoriyel.c · satır 16–20

```
16      int carpim = 1;            /* carpim ← 1 */
17      /* İÇİN i ← 1'DEN n'YE YAP */
18      for (int i = 1; i <= n; i = i + 1) {
19          carpim = carpim * i;   /* carpim ← carpim × i */
20      }                          /* SON */
```

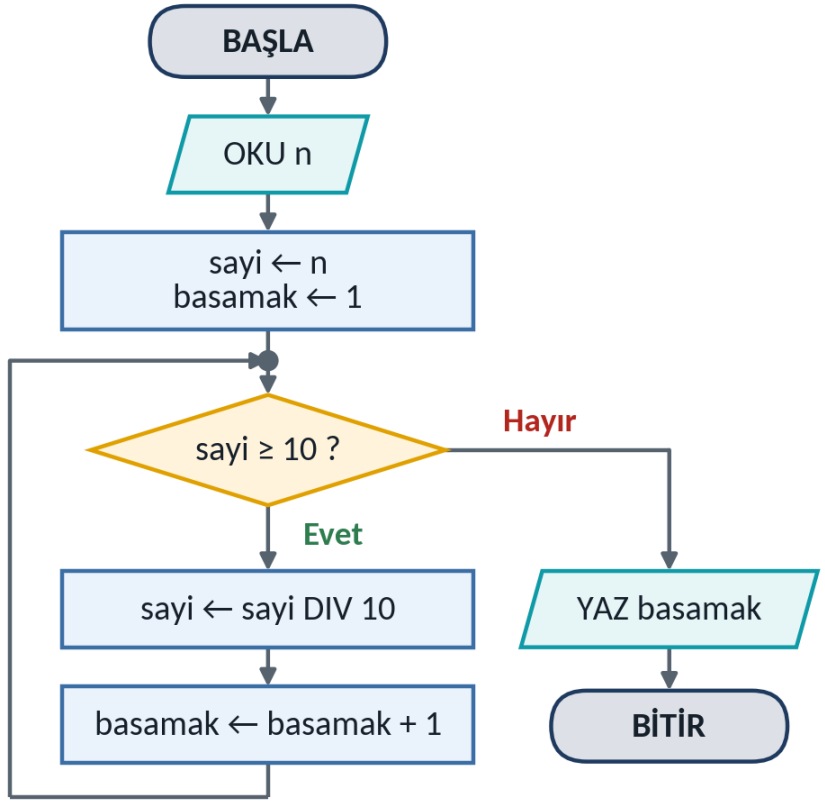


Örnek 11 ★★★ — Basamak sayısı: koşulla biten döngü

SÖZDE KOD

```
BAŞLA
OKU n            // n ≥ 0
sayi ← n
basamak ← 1
İKEN sayi ≥ 10 YAP
    sayi ← sayi DIV 10
    basamak ← basamak + 1
SON
YAZ basamak
BİTİR
```

Tur	sayi ≥ 10 ?	sayi	basamak
başlangıç		**2026**	**1**
1	**2026 ≥ 10 → Evet**	**202**	**2**
2	**202 ≥ 10 → Evet**	**20**	**3**
3	**20 ≥ 10 → Evet**	**2**	**4**
son	**2 ≥ 10 → Hayır**	2	4



Test	n	Bek.	Prog.	
Normal	2026	4	4	✓
Sınır	0	1	1	✓
Sınır	9	1	1	✓
Sınır	10	2	2	✓
Sınır	100	3	3	✓
Kısıt dışı	-305	3	1	✗

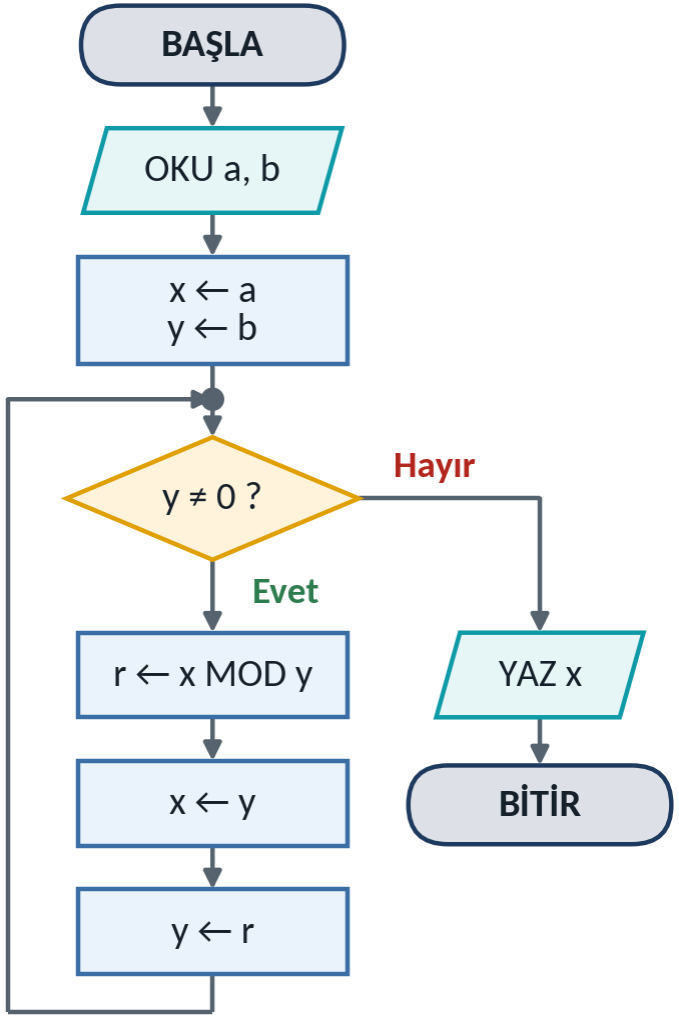
Neden biter? Döngü yalnızca sayi ≥ 10 iken döner ve her turda sayi en az 10 kat küçülür; negatif olmayan tamsayı sonsuza kadar küçülemez. **Son test kaldı:** −305 < 10 → döngü hiç dönmez → “1”. Kısıt (n ≥ 0) neden gerekli, test gösterdi.

Örnek 12 ★★★ — Öklid algoritması ile EBOB

SÖZDE KOD

```
BAŞLA
OKU a, b          // a, b ≥ 0
x ← a
y ← b
İKEN y ≠ 0 YAP
    r ← x MOD y
    x ← y
    y ← r
SON
YAZ x             // EBOB(a, b)
BİTİR
```

Kısıtlar: a, b ≥ 0 ve ikisi birden 0 değil; EBOB(0, 0) tanımsızdır.



Öklid'in gözlemi: a'yı ve b'yi bölen her sayı a MOD b'yi de böler.

EBOB(a, b) = EBOB(b, a MOD b)
EBOB(a, 0) = a

a < b ise? EBOB(18, 48): ilk turda r = 18 MOD 48 = 18, x ← 48, y ← 18. Sayılar kendiliğinden yer değiştirir; **ek karar gerekmez**.

Örnek 12 — izleme tablosu (a = 48, b = 18)

Adım	İşlem	x	y	r	y ≠ 0 ?	Çıktı
1	OKU a, b (a = 48, b = 18)	–	–	–		
2	x ← a	**48**	–	–		
3	y ← b	48	**18**	–		
4	y ≠ 0 ?	48	18	–	**18 ≠ 0 → Evet**	
5	r ← x MOD y	48	18	**12**		
6	x ← y	**18**	18	12		
7	y ← r	18	**12**	12		
8	y ≠ 0 ?	18	12	12	**12 ≠ 0 → Evet**	
9	r ← x MOD y	18	12	**6**		

Adım	İşlem	x	y	r	y ≠ 0 ?	Çıktı
10	x ← y	**12**	12	6		
11	y ← r	12	**6**	6		
12	y ≠ 0 ?	12	6	6	**6 ≠ 0 → Evet**	
13	r ← x MOD y	12	6	**0**		
14	x ← y	**6**	6	0		
15	y ← r	6	**0**	0		
16	y ≠ 0 ?	6	0	0	**0 ≠ 0 → Hayır**	
17	YAZ x	6	0	0		**6**

Kompakt görünüm: (48, 18) → r = 12 · (18, 12) → r = 6 · (12, 6) → r = 0 · y = 0 → EBOB = 6 · 3 tur

Örnek 12 — C önizlemesi, testler ve sonlanma

H02_12_oklid_ebob.c · satır 16–23

```c
    int x = a, y = b;           /* girdilerin kopyaları */
    while (y != 0) {            /* İKEN y ≠ 0 YAP  */
        int r = x % y;          /*   r ← x MOD y   */
        x = y;                  /*   x ← y         */
        y = r;                  /*   y ← r         */
    }                           /* SON             */

    printf("EBOB(%d, %d) = %d\n", a, b, x);
```

Test	a b	Bek.	Prog.	
Normal	48 18	6	6	✓
a < b	18 48	6	6	✓
Aralarında asal	17 5	1	1	✓
b = 0	7 0	7	7	✓
a = 0	0 9	9	9	✓
Büyük sayılar	1071 462	21	21	✓

Neden biter? Yeni y = x MOD y ve 0 ≤ r < y: y her turda **kesin azalan** negatif olmayan bir tamsayıdır → en geç y turda 0 olur.
Neden doğru? Her turda EBOB(x, y) korunur; y = 0 iken EBOB(x, 0) = x yazılır.

Hata Avcısı 1 — başlangıç değeri unutulmuş toplayıcı

✗ HATALI · H02_HA1_ilk_deger_yok.c · satır 11–18

```
11      int n = 5;
12      int toplam;                 /* HATA: toplam ← 0 yok */
13      int i = 1;
14      while (i <= n) {
15          toplam = toplam + i;
16          i = i + 1;
17      }
18      printf("Toplam (1..%d) = %d\n", n, toplam);
```

✓ DOĞRU · H02_07_toplam_dongu.c · satır 15–22

```
15      int toplam = 0;             /* toplam ← 0              */
16      int i = 1;                  /* i ← 1                   */
17      while (i <= n) {            /* İKEN i ≤ n YAP          */
18          toplam = toplam + i;    /*   toplam ← toplam + i   */
19          i = i + 1;              /*   i ← i + 1             */
20      }                           /* SON                     */
21
22      printf("Toplam (1..%d) = %d\n", n, toplam);
```

Gerçek: `gcc -O0` → **uyarı yok**; `gcc -O2` → "'toplam' is used uninitialized". Uyarı olmaması hata olmadığı anlamına gelmez; program çalıştırılmaz (tanımsız davranış).

Hata Avcısı 2 — bir-fark hatası (off-by-one)

✗ HATALI · H02_HA2_bir_fark.c · satır 10-17

```
10      int n = 5;
11      int toplam = 0;
12      int i = 1;
13      while (i < n) {            /* HATA: i ≤ n olmalıydı; n hiç eklenmez */
14          toplam = toplam + i;
15          i = i + 1;
16      }
17      printf("Toplam (1..%d) = %d\n", n, toplam);
```

✓ DOĞRU · H02_07_toplam_dongu.c · satır 15-22

```
15      int toplam = 0;            /* toplam ← 0              */
16      int i = 1;                 /* i ← 1                   */
17      while (i <= n) {           /* İKEN i ≤ n YAP          */
18          toplam = toplam + i;   /*   toplam ← toplam + i   */
19          i = i + 1;             /*   i ← i + 1             */
20      }                          /* SON                     */
21
22      printf("Toplam (1..%d) = %d\n", n, toplam);
```

Gerçek çıktı (hatalı): `Toplam (1..5) = 10` — derleyici uyarısı **yok**. Yalnızca beklenen değeri bilen bir test (n = 5 → 15) ya da izleme tablosu yakalar.

Hata Avcısı 3 — üç sık yanılgı daha

Sonsuz döngü

- `i ← 1, İKEN i ≠ 10 YAP`, gövdede `i ← i + 2`
- i: 1, 3, 5, 7, 9, 11… 10'u atlar
- **Çözüm:** `i ≤ 10`; sayaç koşula yaklaşıyor mu?

Unutulmuş dal

- “a daha büyük / b daha büyük”
- a = b iken yanlış ileti
- **Çözüm:** tüm durumları listele; her dala bir test

Belirsiz adım

- “yeterince”, “biraz”, “uygun”
- Herkes farklı uygular
- **Çözüm:** ölçülebilir yap ya da girdi olarak al

Mühendis Gözüyle ve AP-2 Köprüsü

Mühendis gözüyle

- Doğruluk **ölçülerek** gösterilir: normal, sınır, geçersiz testler.
- Sınırlar hataların evidir: $<$ mı $\leq$ mü?
- Her döngünün **neden bittiğini** gerekçelendir.
- Önce doğru, sonra verimli; verimliliği **ölçerek** karşılaştır.

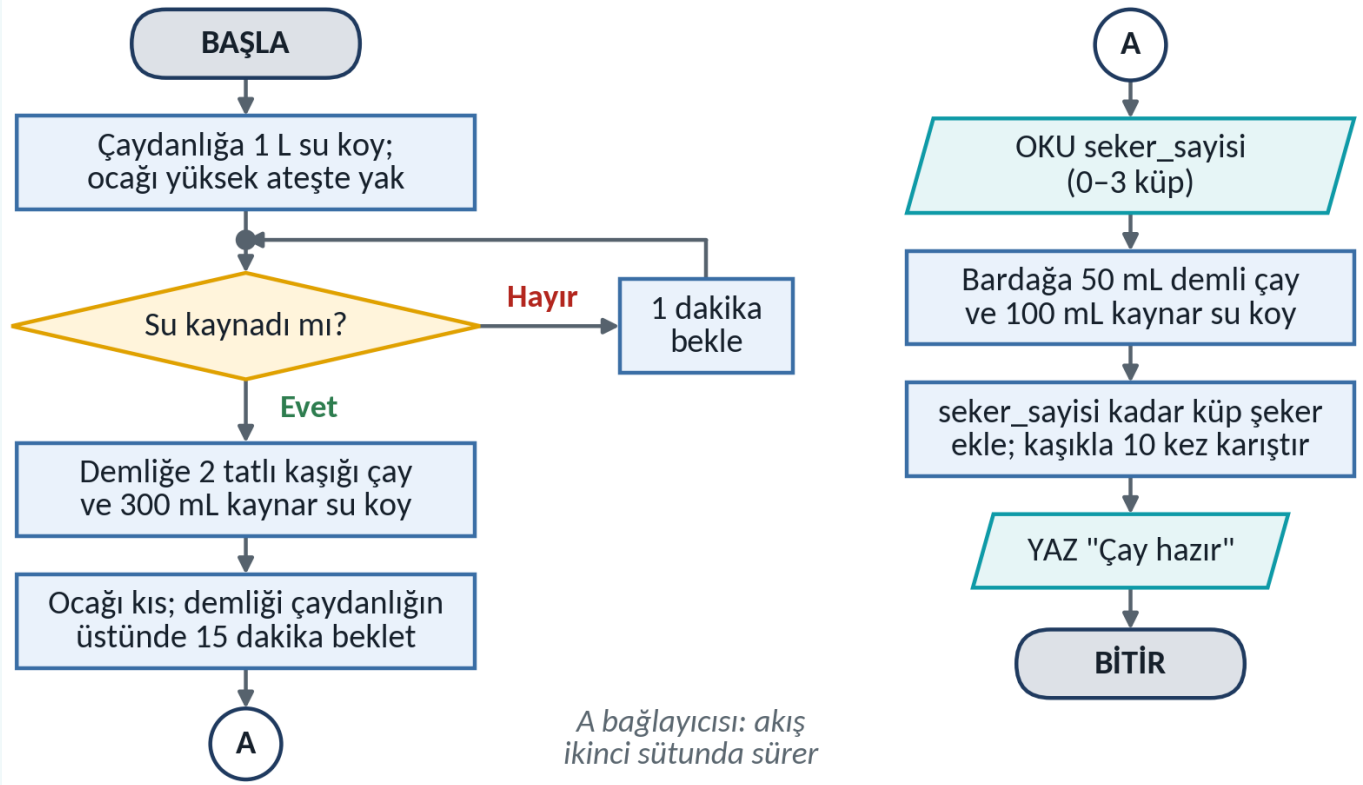
AP-2 köprüsü

- Adım sayma ($3N + 1 \leftrightarrow 3$) → **asimptotik analiz** (O gösterimi)
- "Şimdiye kadarki en büyük" → **seçme sıralaması**
- Öklid → **doğruluk** (döngü değişmezi) ve verimlilik tartışmaları

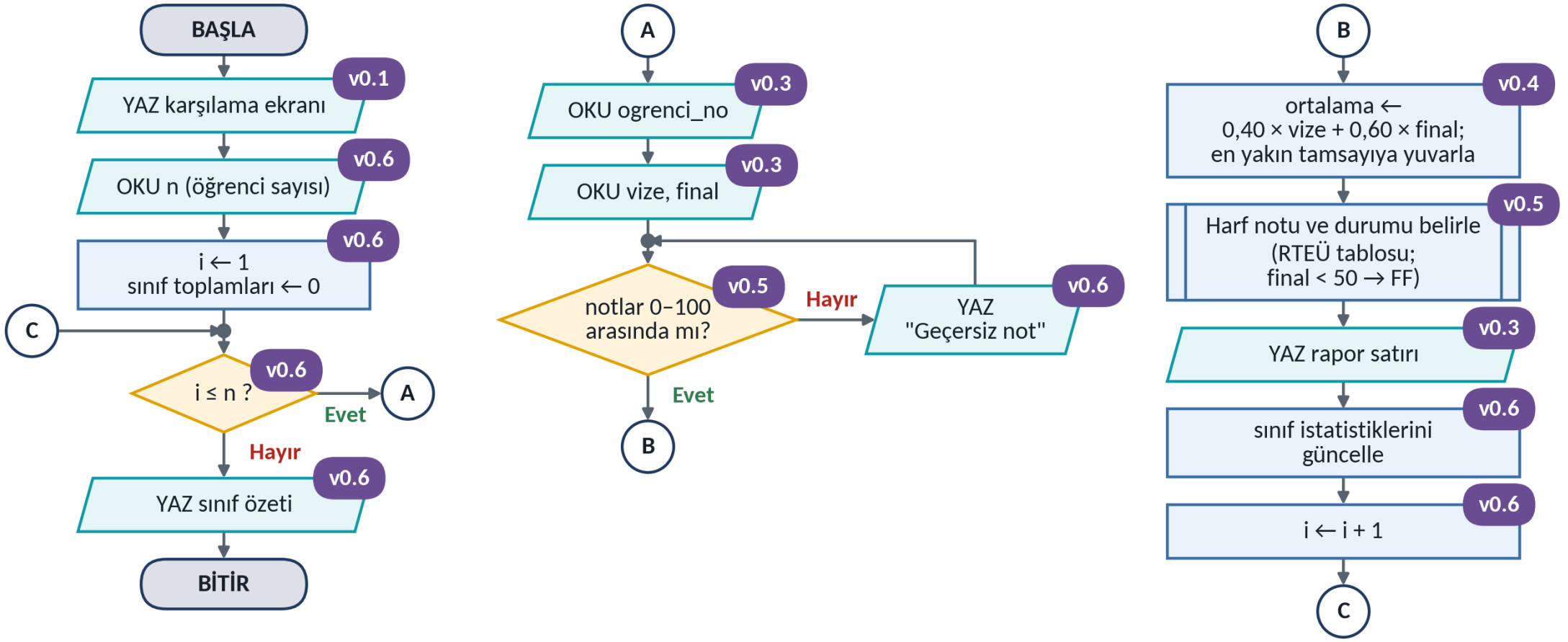
ALGORİTMA ATÖLYESİ

Atölye: robot oyunu (belirsizlik avı)

- Üçlü gruplar: **Programcı · Robot · Gözlemci**.
- Programcı “peynirli sandviç hazırla”yı ≤ 10 adımda yazar.
- Robot **harfiyen** uygular: belirsiz sözcükte “ANLAMADIM: …”, yapılamayan adımda “YAPAMIYORUM” der.
- Gözlemci her duraklamayı not eder; grup algoritmayı yeniden yazar.
- Duraklamaları Knuth'un özellikleriyle eşleştirin.
- Ardından: 5 problem için eşli sözde kod + akış + izleme; Flowgorithm.

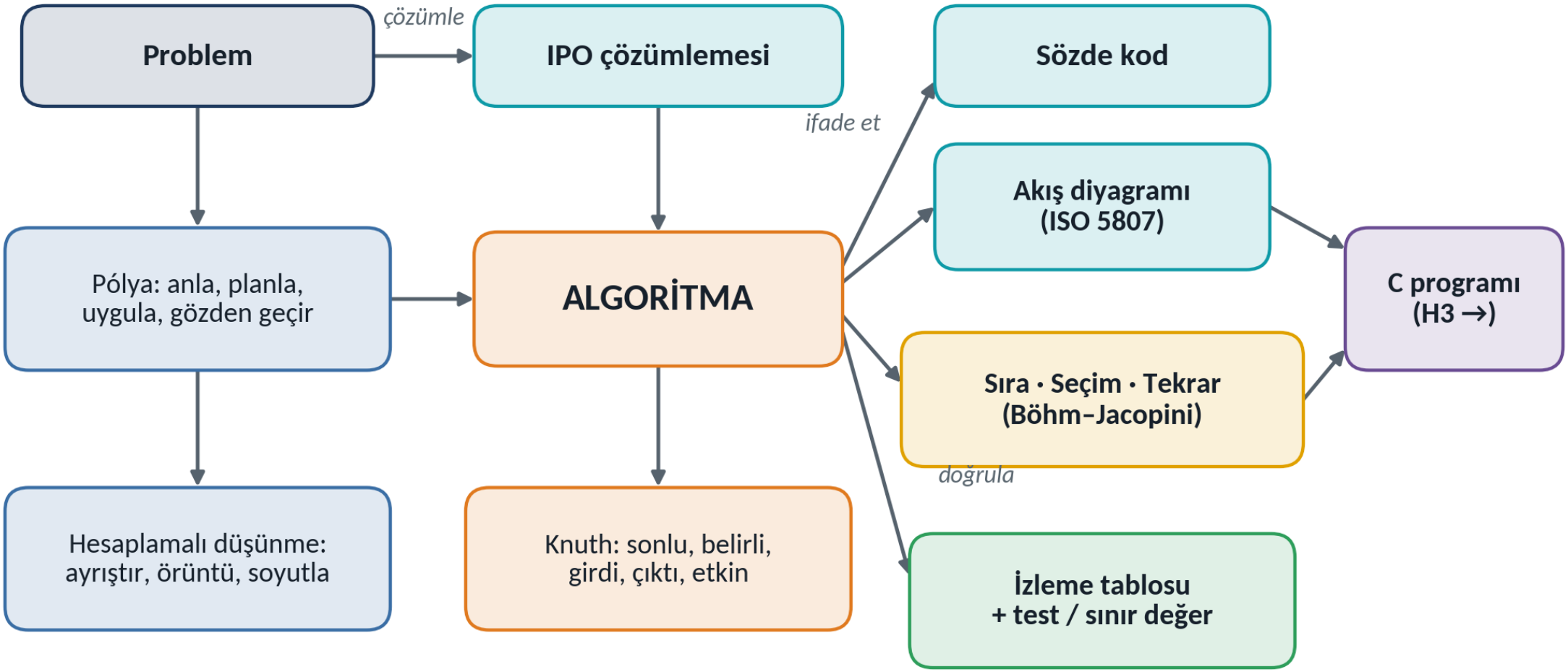


Mini-OBS v0.2: gereksinim çözümlemesi ve ana akış (kod yok)



Kurallar (RTEÜ md. 30): ortalama = 0,40 · vize + 0,60 · final → en yakın tamsayı · dokuz kademeli harf tablosu · final < 50 → FF · CC ve üstü BAŞARILI, DC ŞARTLI BAŞARILI · **mor etiket:** bloğun kodlanacağı sürüm

Özet: haftanın kavram haritası



Problem → çözümle (Pólya, IPO) → algoritma (Knuth) → ifade et (sözde kod, akış) → doğrula (izleme, test) → C (H3)

Büyük Soru'nun yanıtı: “biraz şeker” neden adım değil?

Bilgisayarın “biraz”ı yorumlayacak deneyimi yok: miktarı **kesin** ver ya da **girdi** olarak al. Gizli yapıları (tekrar, seçim) açığa çıkar.

Belirsiz adım	Neden sorunlu?	Algoritmik karşılığı
Biraz su koy	Miktar yok	Çaydanlığa 1 L su koy
Su kaynayınca	Nasıl anlaşılacak?	Kaynadı mı? Hayırsa 1 dk bekle, yeniden bak (tekrar)
Yeterince çay koy	Ölçü yok	Demliğe 2 tatlı kaşığı çay koy
Bir süre demlensin	Süre yok	15 dakika beklet
Biraz şeker ekle	**Kişiye göre değişir (belirlilik!)**	**OKU seker_sayisi (0–3); o kadar küp ekle**
Güzelce karıştır	Ölçülemez	Kaşıkla 10 kez karıştır

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1. “Makarnayı **biraz** tuzlu suda **yeterince** haşla.” Bu adım Knuth'un hangi özelliğini ihlal eder? Nasıl düzeltirsiniz?

2. `EBOB(30, 12)` için Öklid algoritmasının izleme tablosunu kurun. Döngü kaç kez döner?

3. `i ← 1; İKEN i < n YAP toplam ← toplam + i; i ← i + 1 SON` parçası 1'den n'e toplamı neden yanlış hesaplar? Hangi test bunu yakalar?

Haftaya hazırlık ve kaynaklar

- **Oku:** Ders notu (AP1_H02_Ders_Notu.pdf) Bölüm 3–4; web sayfasındaki simülasyonlar (akış diyagramı yürütücüsü, izleme tablosu alıştırması, belirsiz adım oyunu).
- **Yap:** Alıştırma 2.1–2.12'den en az altısı (her türden birer tane); Mini-OBS v0.2 IPO + akış diyagramı.
- **Dene:** kodlar/ klasöründeki önizlemeleri `gcc -std=c17 -Wall -Wextra -pedantic` ile derle ve çalıştır; Flowgorithm'de Örnek 7 ve 12.
- **Gelecek hafta (H3):** değişken kutularının bellekteki gerçek hâli — `int` kaç bayt? `-7` nasıl saklanır? `0.1 + 0.2` neden tam `0.3` değil? 13! neden taşar?

- Pólya (1945) · How to Solve It
- Knuth (1997) · TAOCP Vol. 1, §1.1
- Wing (2006) · Computational thinking
- Böhm & Jacopini (1966)
- ISO 5807:1985 akış diyagramı sembolleri
- Deitel & Deitel (2022) · ana ders kitabı